

Practical #9

Practical Title:

To understand continuous integration concepts and Jenkins architecture for build automation

Theory:

1. What is Continuous Integration (CI)?

Continuous Integration (CI) is a DevOps practice where developers frequently integrate code into a shared repository.

Each integration is automatically tested and built.

Benefits:

- Early bug detection
 - Faster development
 - Improved code quality
-

2. What is Jenkins?

Jenkins is an open-source automation tool used for building, testing, and deploying applications.

It helps implement CI/CD pipelines.

3. Features of Jenkins

- Open-source and free
 - Supports plugins (1000+)
 - Easy integration with Git, Docker, etc.
 - Automation of build and testing
 - Distributed builds (master-agent architecture)
-

4. Jenkins Architecture

Jenkins architecture consists of:

- **Master (Controller):**
 - Manages jobs and scheduling
 - Handles UI and configurations
- **Agent (Slave):**
 - Executes build tasks
 - Can run on multiple machines

Flow:

Developer → GitHub → Jenkins → Build → Test → Deploy

Procedure:

Step 1: Install Jenkins and start server

Step 2: Access Jenkins Dashboard (<http://localhost:8080>)

Step 3: Create a new job (Freestyle Project)

Step 4: Configure source code management (Git repository)

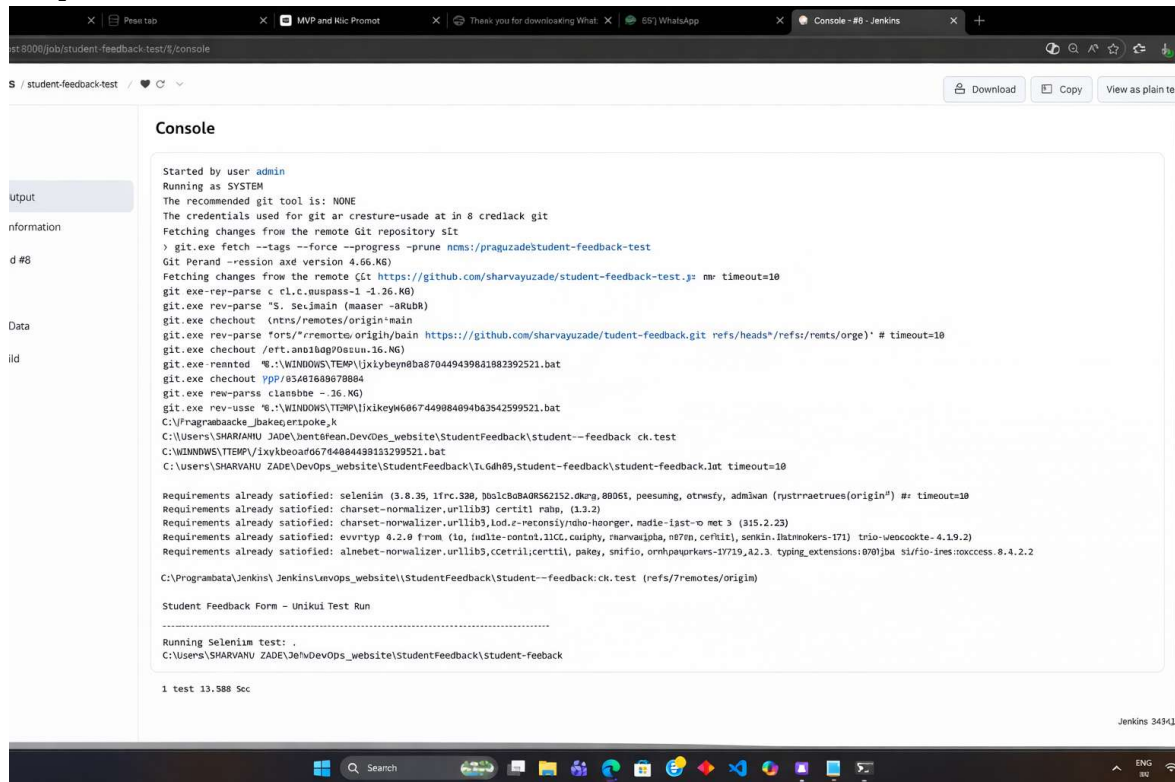
Step 5: Configure build trigger (Poll SCM / Webhook)

Step 6: Add build steps (Execute shell / commands)

Step 7: Build the project

Step 8: View console output

Output Screenshots:



The screenshot shows a Jenkins console window for a job named 'student-feedback-test'. The output text is as follows:

```
Started by user admin
Running as SYSTEM
The recommended git tool is: NONE
The credentials used for git are: crestore-usade at in 8 credial git
Fetching changes from the remote Git repository sit
> git.exe fetch --tags --force --progress --prune ncms://praguzade/student-feedback-test
Git Perand --ession axd version 4.66.KG)
Fetching changes from the remote Git repository https://github.com/sharvayuzade/student-feedback-test.: m timeout=10
git.exe rev-parse c cl.c.nuspass-1 -1.26.KG)
git.exe rev-parse "S. SeJmain (maaser -aRuBk)
git.exe checkout (ncms/remotes/origin/main
git.exe rev-parse forst/remotes/origin/bain https://github.com/sharvayuzade/tudent-feedback.git refs/heads*/refs/remts/orge) # timeout=10
git.exe checkout /ert.ano18ap00scuu.16.KG)
git.exe reentod %..\\WINDOWS\\TEMP\\Jjkiybeyn8ba8704494398a1882392521.bat
git.exe checkout ypp/83487688678884
git.exe rev-parse clansbbe -.26.KG)
git.exe rev-usse %..\\WINDOWS\\TEMP\\Jjkiybeyn8ba8704494398a1882392521.bat
C:\\ProgramData\\Jenkins\\Jenkins\\tmp\\Jjkiybeyn8ba8704494398a1882392521.bat
C:\\Users\\SHARVANI\\JADE\\DevOps_website\\StudentFeedback\\student-feedback ck.test
C:\\WINDOWS\\TEMP\\Jjkiybeyn8ba8704494398a1882392521.bat
C:\\Users\\SHARVANI\\ZADE\\DevOps_website\\StudentFeedback\\T.L.GdH85,student-feedback\\student-feedback.1st timeout=10

Requirements already satisfied: selenium (3.8.39, ifrc.338, 90a1c80a40562152.dkrp, 88061, peesumng, otrwsfy, admixan (nysttraetrues(origin?) #=: timeout=10
Requirements already satisfied: charset-normalizer.urllib3 certifi raba, (1.3.2)
Requirements already satisfied: charset-normalizer.urllib3, lod.e-reconsi/ndio-heorger, nadie-1ast-o met 3 (315.2.23)
Requirements already satisfied: evrrtyp 4.2.0 from (lo, twdite-contol.11CC, carlphy, rnarvaipha, n87m, certifi, semkin, llatnokers-171) trio-weocokite- 4.19.2)
Requirements already satisfied: alnebet-normalizer.urllib3, ccetrl;certifi, pakey, smfio, ornhpaprkrars-17719, 42.3. typing_extensions: 0707ba si/fo-ines:toaccess.8.4.2.2

C:\\ProgramData\\Jenkins\\Jenkins\\tmp\\Jjkiybeyn8ba8704494398a1882392521.bat
Student Feedback Form - Unikui Test Run
-----
Running Selenium test: -
C:\\Users\\SHARVANI\\ZADE\\DevOps_website\\StudentFeedback\\student-feedback

1 test 13.588 Sec
```

Activity / Task:

Task 1: Jenkins Job

Repository Link: https://github.com/sharvayuzade/DevOps_CA2

Task 2: Jenkins Pipeline Concept

Jenkins Pipeline is a set of automated steps that define the CI/CD process.

Types:

- **Declarative Pipeline** (simple syntax)
- **Scripted Pipeline** (advanced scripting)

Example flow:

- Code → Build → Test → Deploy

Written in:

- **Jenkinsfile**
-

Task 3: Steps to Integrate GitHub with Jenkins

1. Create repository on GitHub
2. Copy repository URL
3. In Jenkins → Configure job → Add Git URL
4. Add credentials (if private repo)
5. Enable Webhook in GitHub:
 - Go to repo → Settings → Webhooks
 - Add URL:

`http://your-jenkins-url/github-webhook/`

6. Select trigger in Jenkins:
 - “GitHub hook trigger for GITScm polling”
 7. Save and test
-

Advantages

1. Automates build and testing
 2. Faster development cycle
 3. Early bug detection
 4. Supports CI/CD pipelines
 5. Highly extensible (plugins)
-

Challenges

1. Initial setup complexity
2. Plugin management issues
3. Requires maintenance

